

## Bài 5 - Chương 3

### CÁC DẠNG THUẬT TOÁN CƠ BẢN

#### 3.1. THUẬT TOÁN CẤU TRÚC

Trong quá trình phát triển tin học, nghệ thuật lập trình không ngừng được cải tiến, nhiều phương pháp lập trình mới được đề nghị như: thiết kế thuật toán và chương trình theo mô đun, lập trình có cấu trúc, cách tiếp cận từ trên xuống (top - down), kỹ thuật đi từ dưới lên (bottom - up), chia để trị (divide and conquer) v.v.... Khi giải quyết các bài toán lớn, xây dựng thuật toán cho chúng thường phải kết hợp đồng thời các kỹ thuật trên.

##### 3.1.1. Thiết kế theo mô đun.

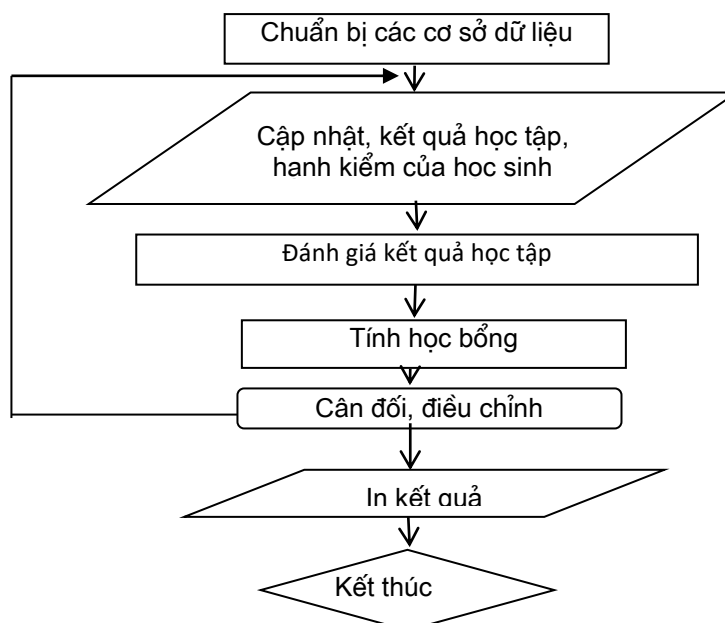
Với những bài toán lớn, hệ thống lớn, khi xây dựng thuật toán ta thường phải chia nhỏ ra thành từng khối, mỗi khối thực hiện một số nhiệm vụ xử lý nhất định. Khi chuyển thành chương trình, mỗi khối này sẽ là một mô đun — hàm (function). Đó chính là tổng của kỹ thuật lập trình theo mô đun. Mỗi mô đun có những đặc trưng sau:

- Thực hiện trọn gói một số nhiệm vụ xử lý tin cụ thể;
- Chỉ có một lối vào và một lối ra;
- Có thể ghép nối vào trong quá trình xử lý tin khác mà không cần thay đổi gì trừ thay đổi dữ liệu ban đầu;
- Có thể gọi, sử dụng các mô đun khác.

Lập trình theo mô đun có lợi ở chỗ các mô đun có thể giao cho các nhóm lập trình, sau đó ghép lại trong một chương trình quản lý chung, mỗi mô đun có độ phức tạp vừa phải cho phép dễ theo dõi, sửa chữa, bổ sung.

Ví dụ Sử dụng kỹ thuật thiết kế mô đun để xây dựng chương trình quản lý đào tạo. Đây là một bài toán khá phức tạp, bao gồm nhiều phần việc, chúng được tổ chức theo các mô đun trong sơ đồ khối như trên hình sau.

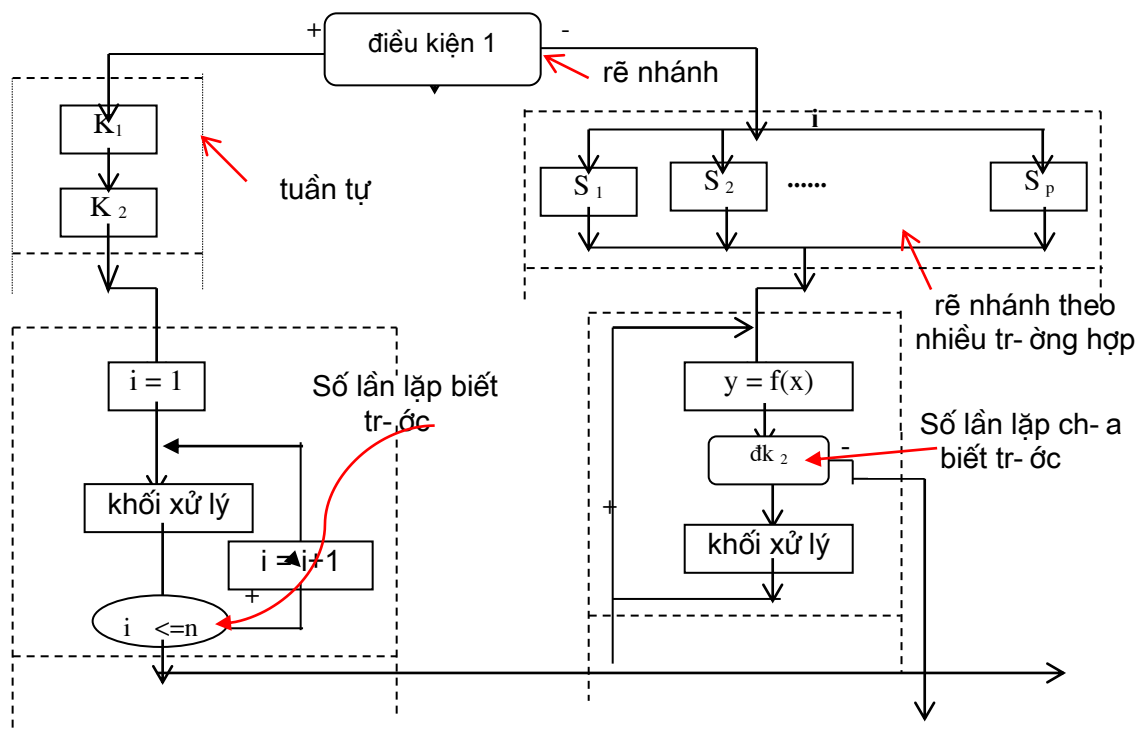
Lập trình theo mô đun cho phép tổ chức tính toán một cách tối ưu. Các nhiệm vụ tính toán, xử lý khác nhau được tổ chức thành các mô đun khác nhau, mỗi mô đun là một chương trình con, khi cần tính toán hay xử lý chỉ việc gọi nó. Chúng ta cũng có thể sửa đổi, bổ sung các mô đun mà không ảnh hưởng đến toàn bộ chương trình lớn.



### 3.1.2. Thuật toán có cấu trúc

Mỗi mô đun thuật toán thường gồm nhiều thủ tục-hàm tính toán, xử lý. Diễn đạt các thủ tục-hàm xử lý tin một cách rõ ràng, logic, dễ phát hiện sai sót, dễ khẳng định tính đúng đắn của thuật toán là điều quan trọng trước khi viết chương trình. Dijkstra E.W đã đề xuất một phương pháp hữu hiệu, giúp cho vẽ sơ đồ thuật toán có hiệu quả, đó là phương pháp lập trình có cấu trúc. Theo quan điểm lập trình cấu trúc, các thuật toán được phân ra làm 3 dạng cấu trúc cơ bản: tuần tự, rẽ nhánh đơn giản + rẽ nhánh theo nhiều trường hợp, xử lý lặp(loop) hay thường gọi là thuật toán chu trình; thuật toán của bài toán lớn được chia thành nhiều khối thuộc một trong ba cấu trúc cơ bản trên và chỉ có một lối vào một lối ra.

Hình 2.2 giới thiệu một mô đun thuật toán có chứa các cấu trúc cơ bản.



Hình 3.2

Để cho mỗi khối có tính độc lập, trong nó phải có đủ ba thành phần : chuẩn bị dữ liệu ban đầu, phần tính toán, phần kết thúc khối. Lập trình cấu trúc hạn chế tối đa việc chuyển điều khiển đến các mốc(nhãn) để thoát ra giữa chừng mà không qua điểm kết thúc khối, chỗ nào có nhãn phải thay bằng một khối cấu trúc. Sau đây sẽ xét các dạng thuật toán cơ bản đã nêu.

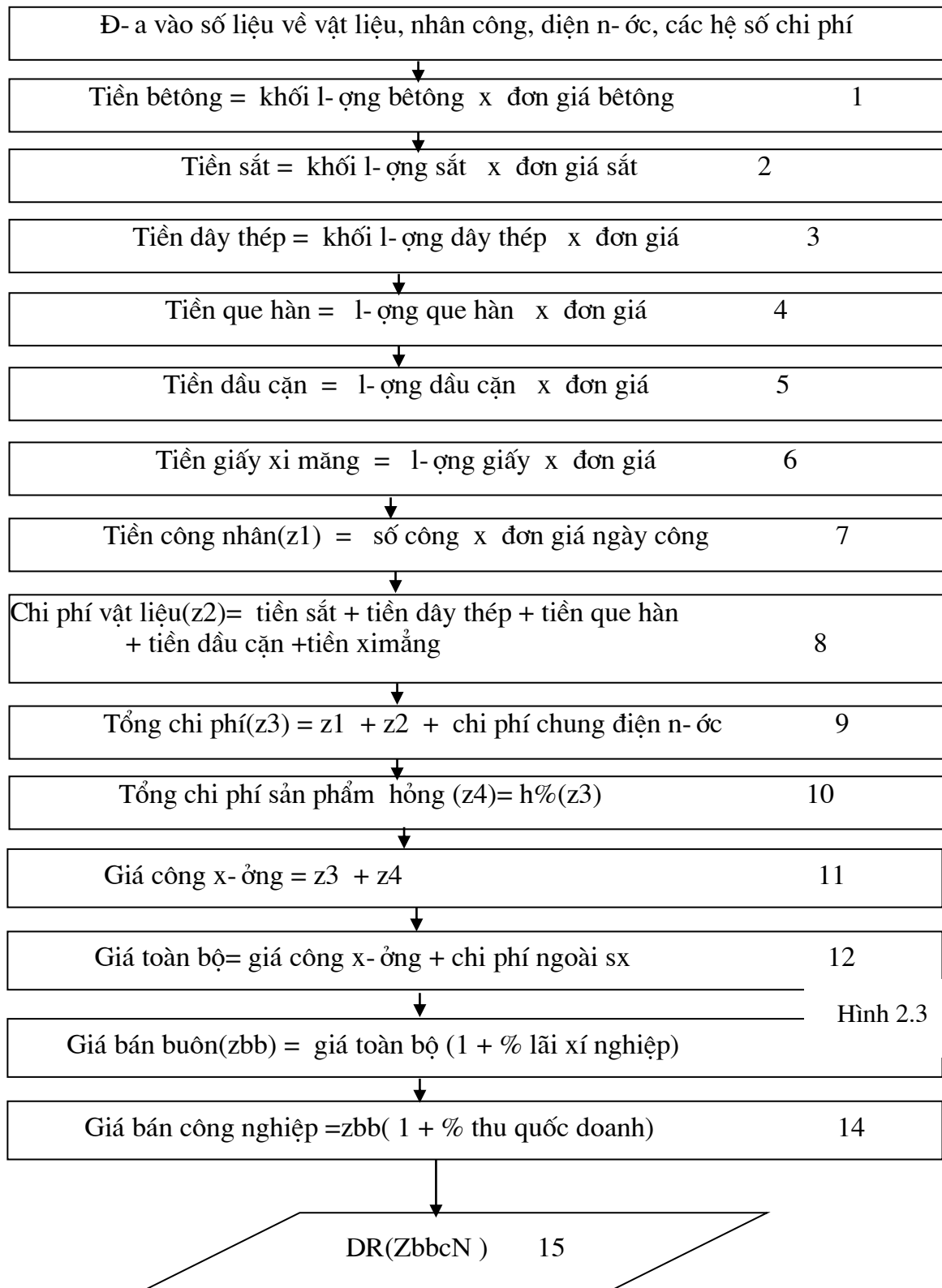
## 3.2. THUẬT TOÁN CẤU TRÚC BÀI TOÁN TUẦN TỰ

Ví dụ .Giá bán một sản phẩm bê tông cấu thành từ các khoản mục chi phí

- bê tông;- sắt;- que hàn;- dây thép;- giấy xi măng;- dầu cặn;- điện nước;- chi phí sản phẩm hàng;- chi phí ngoài sản xuất(%);- chi phí công x-ông(%);- nộp thu quốc doanh(%);- nhân công;

Mỗi loại sản phẩm bê tông có một con số riêng về chi phí từng loại vật liệu ( số lượng, chủng loại, giá cả) về điện nước, về các loại chi phí nh- nhân công,... Tính giá cho một sản phẩm bê tông là tính chi phí cụ thể cho từng khoản mục sau đó cộng lại.

Ta có sơ đồ sau:



Trong sơ đồ trên, quá trình tính toán đi tuần tự từ khối đ- a dữ liệu vào, qua các khối số 1,2..... cho đến 15, không phải rẽ nhánh, không tính lặp. Dạng sơ đồ tuần tự nh- vậy còn đ- ọc gọi là thuật toán đơn giản nhất.

### 3.3. THUẬT TOÁN RẼ NHÁNH ỚN GIỎN

Dạng tuần tự th-ờng chỉ là một đoạn, một b-ớc nào đó trong cả bài toán, còn lại phần lớn là thuật toán đều chứa một hay nhiều phần rẽ nhánh. Sơ đồ khối trong hình 2.4 cho ta một hình ảnh rõ ràng về sự rẽ nhánh. Trong đó, nếu  $0 \leq x \leq c$  đúng (+) thì quá trình tính toán sẽ theo phía trái ng-ợc lại (-), rẽ sang phải .

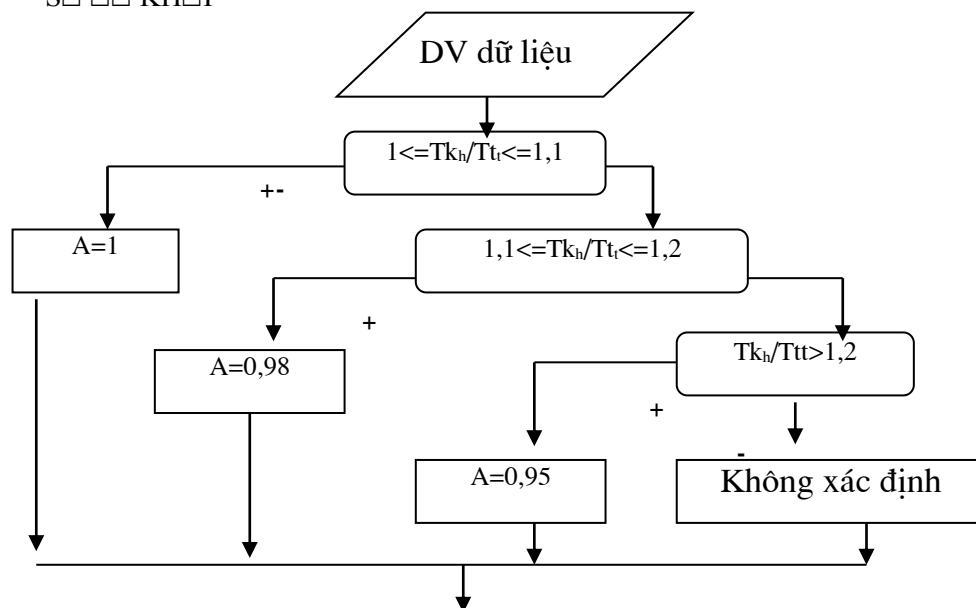
Ví dụ Hệ số tăng hiệu quả sử dụng vốn đầu t- vào thiết bị đượctính theo công thức:

$$K_c = 1 + A \frac{T_{kh} - T_{tt}}{T_{tt}} + L(K_{lkh} - K_{ltt}) - \frac{Mt - M_{tt}}{Mt}$$

trong đó hệ số A sẽ nhận giá trị theo điều kiện sau:

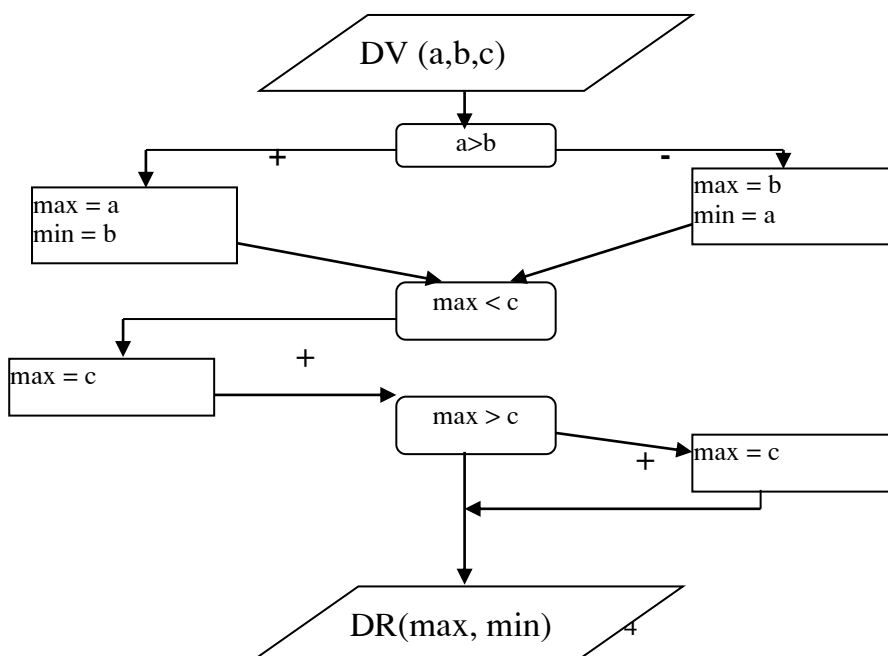
$$A = \begin{cases} 1 & \text{neu } 1 \leq T_{kh}/T_{tt} \leq 1,1 \\ 0,98 & \text{neu } 1,1 \leq T_{kh}/T_{tt} \leq 1,2 \\ 0,95 & \text{neu } T_{kh}/T_{tt} > 1,2 \end{cases}$$

SƠ ĐỒ KHỐI



Hình 3.4

Trong ví dụ trên, vì A nhận từng giá trị phụ thuộc vào từng khoản giá trị của  $T_{kh}/T_{tt}$  nên ta phải tiến hành các khối kiểm tra cụ thể, chứ không thể kiểm tra một lần. Nếu trong bài toán chỉ có hai khả năng rẽ nhánh thì chỉ phải kiểm tra một lần



Ví dụ . Cho ba số a, b, c lập sơ đồ thuật toán chọn số nhỏ nhất và lớn nhất của chúng. Thuật toán có dạng nh- trên hình trên

### 3.4. THUẬT TOÁN RẼ NHÁNH THEO NHIỀU TRƯỜNG HỢP

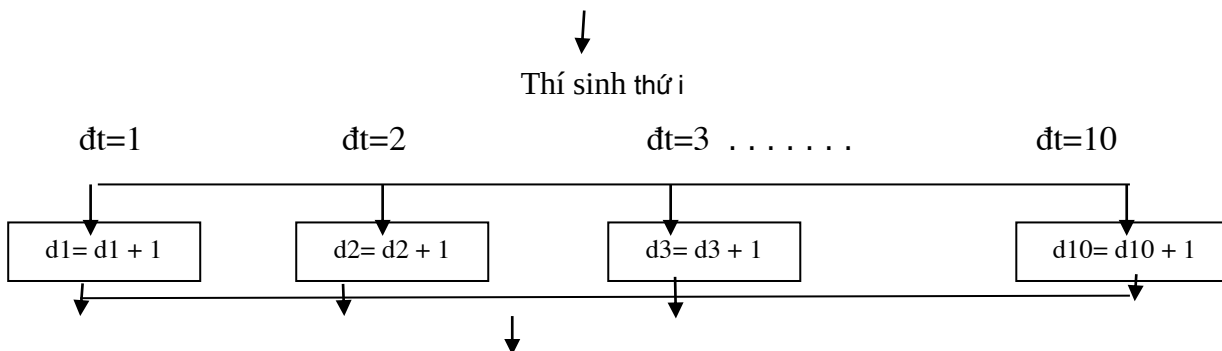
Ví dụ 3.5. Có m-ời đối t-ợng thí sinh dự thi, mỗi thí sinh chỉ đăng ký một đối t-ợng:

| TT | Họ và tên | Đối t-ợng |
|----|-----------|-----------|
| 1  | H.B.A     | 9         |
| 2  | H.T.Q.A   | 2         |

Lập sơ đồ khối để thống kê mỗi loại đối t-ợng có bao nhiêu thí sinh. Để vẽ thuật toán rẽ nhánh theo nhiều tr-ờng hợp cho ví dụ này, ta làm nh- sau

- Đặt biến dt( đối t-ợng) để kiểm tra rẽ nhánh; Tạo 10 bộ đếm d1=0,..., d10=0 (lúc đầu gán = 0) để mỗi lần so sánh giá trị trong cột đối t-ợng, sẽ tăng dk lên một đơn vị nếu dt bằng k

Tại mỗi dòng (mỗi thí sinh) phải thực hiện công việc kiểm tra và rẽ nhánh theo tr-ờng hợp nh- trên hình 3.6



Hình 3.6

Trong ngôn ngữ lập trình C++, rẽ nhánh theo nhiều tr-ờng hợp đ-ợc thực hiện bởi câu lệnh switch case

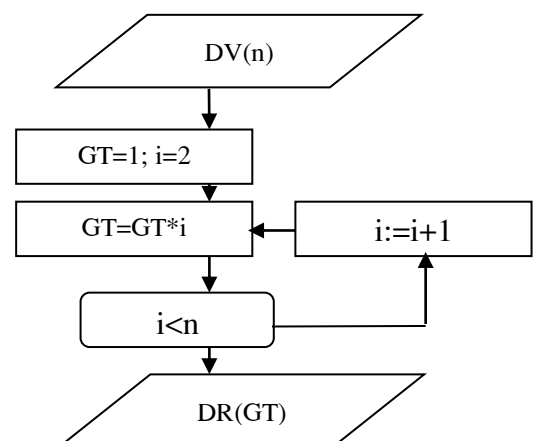
### 3.5. THUẬT TOÁN BÀI TOÁN LẬP

Ta có thể chia bài toán lập làm hai loại: có số lần lặp biết tr-ớc và có số lần lặp ch-a biết tr-ớc. Trong mỗi loại đó lại chia ra các dạng: dạng tính tổng, dạng tính tích, dạng tính khác tổng và tích.

#### 3.5.1 lập tính tích có số lần lặp biết tr-ớc

Ví dụ. Tính giai thừa của n:  $n! = 1.2.3... (n-1)n$ . Phân tích bài toán này ta thấy, nếu ban đầu ta đặt  $GT = 1$  sau đó sẽ tính lặp công thức  $GT = GT * i$ , bắt đầu  $i = 2$  cho đến  $n$  thì kết quả sẽ cho  $n!$ . sơ đồ khối nh- ở hình 3.7.

Đặc điểm của chu trình tính tích là giá trị ban đầu của tích đ-ợc gán bằng 1.

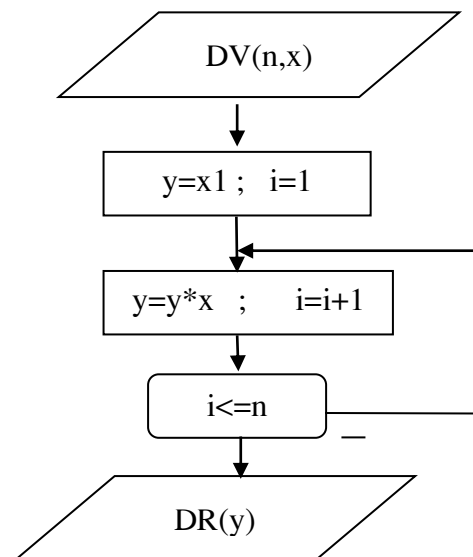


hình 3.7

Trong sơ đồ chu trình, phải xác định đ-ợc công thức tính toán chính mà sẽ đ-ợc tính lặp ( ở đây là  $GT=GT*i$ ); biến  $i$  ở đây gọi là tham số điều khiển chu trình ; ( $i < n$ ) là kiểm tra điều kiện kết thúc chu trình

Ví dụ Tính  $y = x^n$  hay

$$y = \underbrace{x \cdot x \cdot \dots \cdot x}_n$$



Hình 3.8

Ta tổ chức nh- sau: có 4 đại l- ợng tham gia giá trình tính:  $y$ ;  $x$ ;  $n$  và một số đếm chu trình  $i$ , lúc đầu gán  $x$  cho  $y$ , ( $y=x$ );  $i$  nhận giá trị từ hai ( $i=2$ ) sau đó sẽ tính lặp công thức  $y=y*x$ . Mỗi lần tính công thức này, lại tăng bộ đếm  $i$  lên một đơn vị ( $i=i+1$ ) quá trình này lặp cho đến khi  $i > n$ . Sơ đồ khối cho đến hình 3.8

### 3.5.2 Chu trình tính tổng có số lần lặp biết tr-ớc

Ví dụ 3.8. Lập sơ đồ thuật toán tính tổng dãy số thực :

$$S = \sum_{i=1}^n a_i$$

Bài toán trên đ-ợc phân tích nh- sau

$$S = \underbrace{a_1 + a_2}_{s1} + a_3 + \dots + a_{n-1} + a_n$$

$\underbrace{\hspace{1cm}}_{s1}$

$\underbrace{\hspace{1cm}}_{s2} \dots \dots \dots$

$\underbrace{\hspace{1cm}}$

$S_{n-1}$

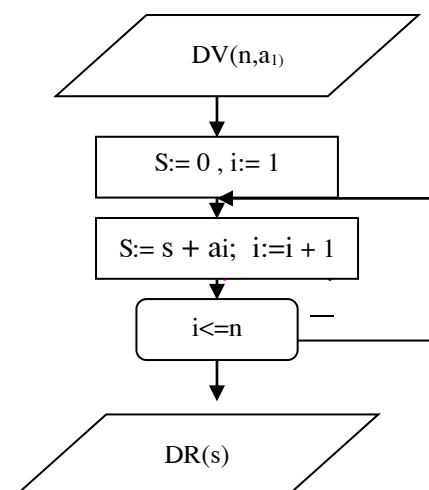
Lúc đầu  $S=0$ .

Khi  $i=1$ ,  $S = S + a_i = a_1$

$i=2$ ,  $S = S + a_i = a_1 + a_2$

...

$i=n$ ,  $S = S + a_i = a_1 + a_2 + \dots + a_n$ .



Công thức tính lặp là  $S = S + a_i$ . Sơ đồ khối nh- trên hình 3.9

Đặc điểm của sơ đồ thuật toán tính tổng là giá trị ban đầu của tổng gán bằng 0 (hoặc có thể gán giá trị ban đầu của tổng bằng phân tử đầu tiên của dãy). Các phần khác cũng giống sơ đồ tính tích.

Ví dụ 3.9. . Cho n giá trị thống kê- lập sơ đồ thuật toán tính các giá trị:

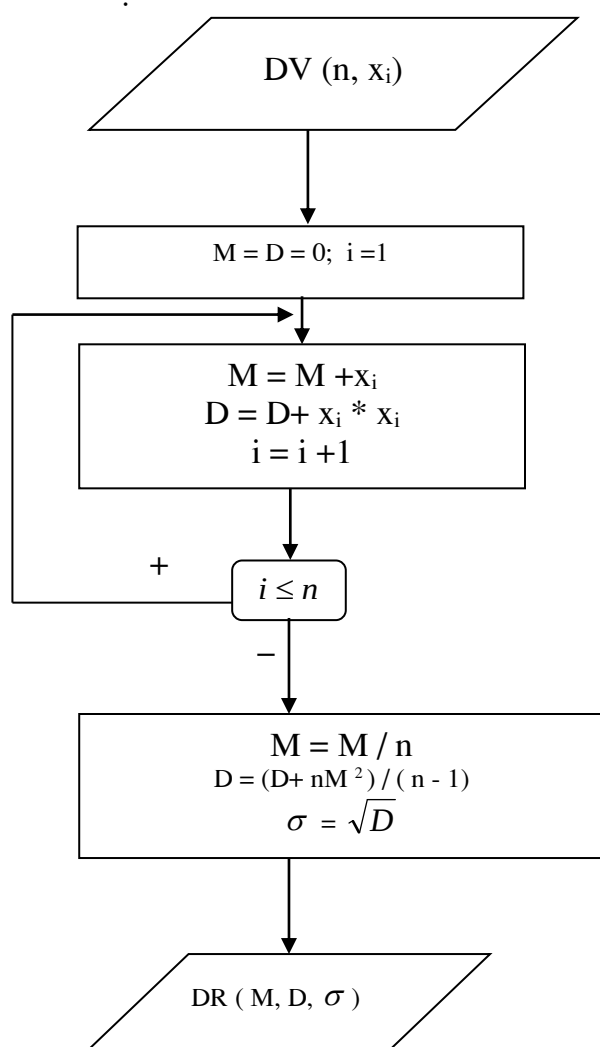
- kỳ vọng toán học :  $M = \frac{1}{n} \sum_{i=1}^n x_i$  ;

- Ph- ơng sai

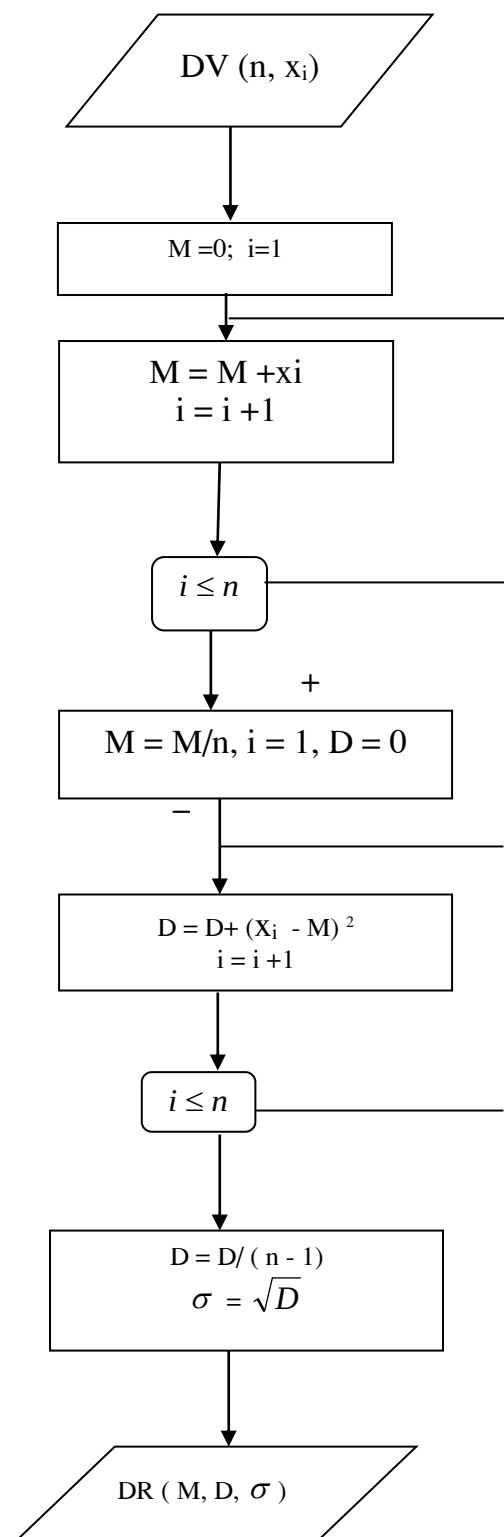
$$D = \frac{1}{n-1} \sum_{i=1}^n (x_i - M)^2 ;$$

- Độ lệch trung bình bình ph- ơng  $\sigma = \sqrt{D}$ .

Sơ đồ thuật toán trên hình 3.10.



Hình 3.10



Hình 3.11

Sơ đồ trên có thể cải tiến hành sơ đồ đơn giản hơn ( h . 3.11)

bằng các biến đổi sau:

$$D = \frac{1}{n-1} \sum_{i=1}^n (x_i - M)^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i^2 - 2x_i M + M^2) = \frac{1}{n-1} \sum_{i=1}^n x_i^2 - 2M \sum_{i=1}^n x_i + \sum_{i=1}^n M^2$$

$$= \frac{1}{n-1} \sum_{i=1}^n x_i^2 - nM^2.$$

Ví dụ 3.10

Tính tích phân xác suất

$$p(x) = \frac{1}{\sqrt{2\pi}} \int_{-x}^x e^{-t^2/2} dt; \quad (3.1)$$

Trước hết, ta phân tích để lập sơ đồ thuật toán tính một tích phân định hạn tổng quát

$$I = \int_a^b f(x) dx \quad (3.2)$$

Từ toán giải tích, ta đã biết công thức:

$$I = \int_a^b f(x) dx = h \sum_{i=0}^{n-1} f(x_i); \quad (3.3)$$

trong đó h là b-ớc chia,  $h = (b-a)/n$ .

Triển khai công thức (3.3) (tính tích phân theo phương pháp hình thang.)

$$I = 0.5(f(a) + f(a+h)) + 0.5 [ (f(a+h) + f(a+2h)) + \dots + (f(a+(n-1)h) + f(b)) ] h$$

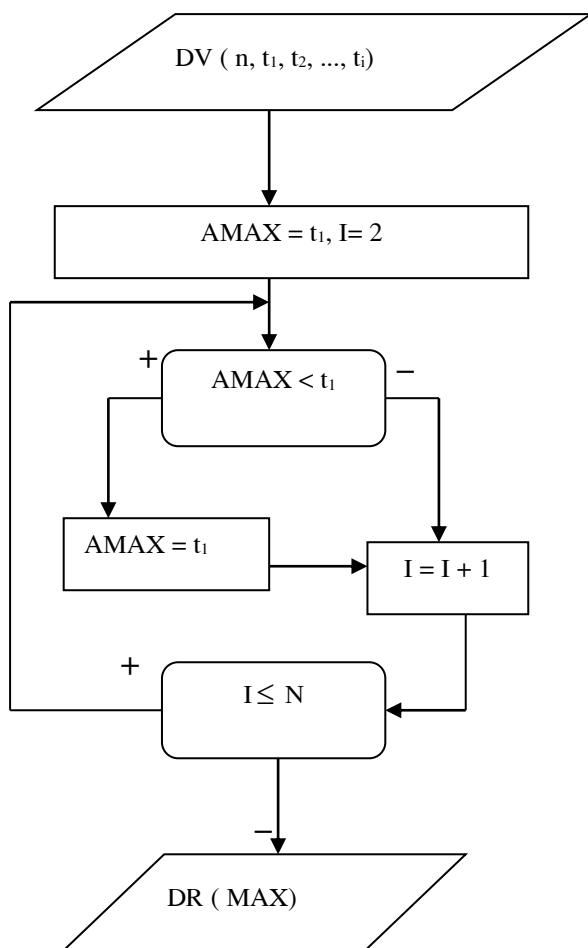
Gộp những phân tử giống nhau ta có

$$I = 0.5 \left[ f(a) + f(b) + h \sum_{i=1}^{n-1} f(a+ih) \right]; \quad (3.4)$$

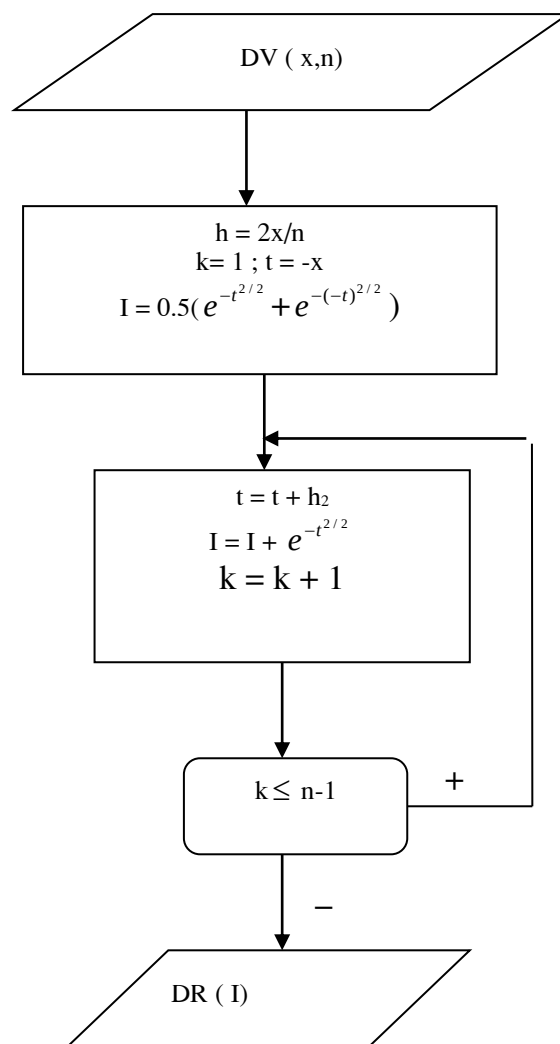
Thay (3.4) vào (3.1) ta sẽ có công thức tính

$$P(x) = 0.5 \left[ (e^{-(x)^2/2} + e^{-(-x)^2/2}) + \sum_{i=1}^{n-1} e^{-(x+ih)^2/2} \right] \frac{h}{\sqrt{2\pi}}$$





Hình 3.13



Hình 3.12

Sơ đồ khối của bài toán trên hình 3.12

### 3.5.3. Thuật toán của bài toán lặp phân nhánh, lồng nhau

Ví dụ 3.11 Lập sơ đồ thuật toán tính  $\max(t_1, t_2, \dots, t_i)$  với dãy  $t_1, t_2, \dots, t_i$  cho trước. Để làm bài toán này ta phải sử dụng một biến AMAX để giữ lấy giá trị lớn nhất của dãy số  $t_1, t_2, \dots, t_i$ . Lần thứ nhất ta gán  $t_1$  cho AMAX, sau đó so sánh AMAX với  $t_2$ . Nếu AMAX bé hơn  $t_2$ , thì gán  $t_2$  cho AMAX nếu không thì không làm gì cả, chuyển tiếp so sánh AMAX với  $t_3$ , v.v... Quá trình lặp lại cho đến khi đã duyệt hết dãy số. Sơ đồ thuật toán cho trên hình 3.13. Như ta thấy, ở đây vừa có chu trình, vừa có rẽ nhánh. Cứ sau mỗi lần lặp, Biến AMAX sẽ chứa giá trị lớn nhất của những  $t_1, t_2, \dots, t_i$  đã duyệt.

Ta chỉ giữ giá trị lớn nhất chứ không để ý là giá trị lớn nhất đó đạt được bởi bằng bao nhiêu. Theo cách làm này, ở lần lặp thứ  $n$  (cuối cùng) trong AMAX sẽ chứa giá trị lớn nhất của toàn dãy số

Ví dụ 3.12 Cho ma trận vuông:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ \dots & \dots & \dots & \dots \\ a_{1n} & a_{nn} & \dots & a_{nn} \end{bmatrix}$$

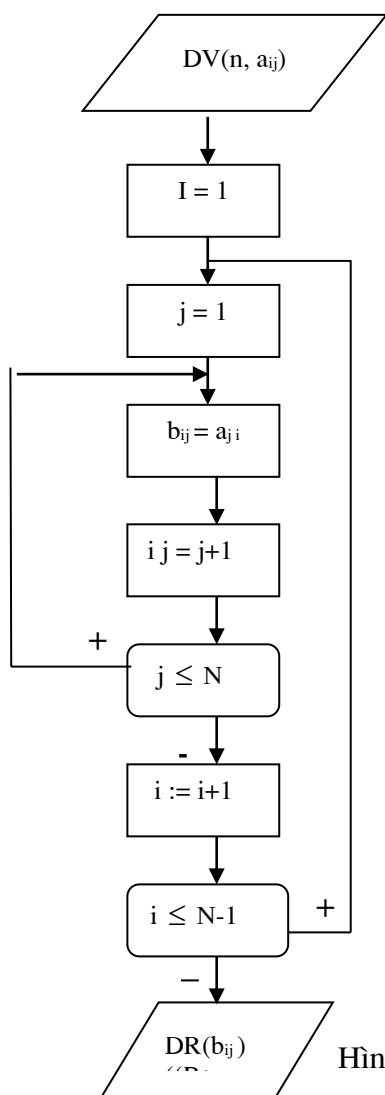
Lập sơ đồ thuật toán hoán vị ma trận A. Ma trận kết quả A có các phần tử  $b_{ij} = a_{ji}$ . Ma trận có hai chiều, các phần tử trên dòng chạy theo chỉ số j, trên cột chạy theo chỉ số i, mỗi chiều đều biến đổi từ 1 đến n. Hoán vị ma trận thực chất là hoán vị dòng cho cột và ngược lại, phần tử ở dòng i cột j, sẽ trở thành phần tử cột i và dòng j trong ma trận kết quả. Sơ đồ có hai chu trình lồng nhau (h.3.14).

Ví dụ 3.13 Xét ví dụ có ba chu trình lồng nhau: nhân hai ma trận. Cho hai ma trận A, B ban đầu và ma trận C kết quả

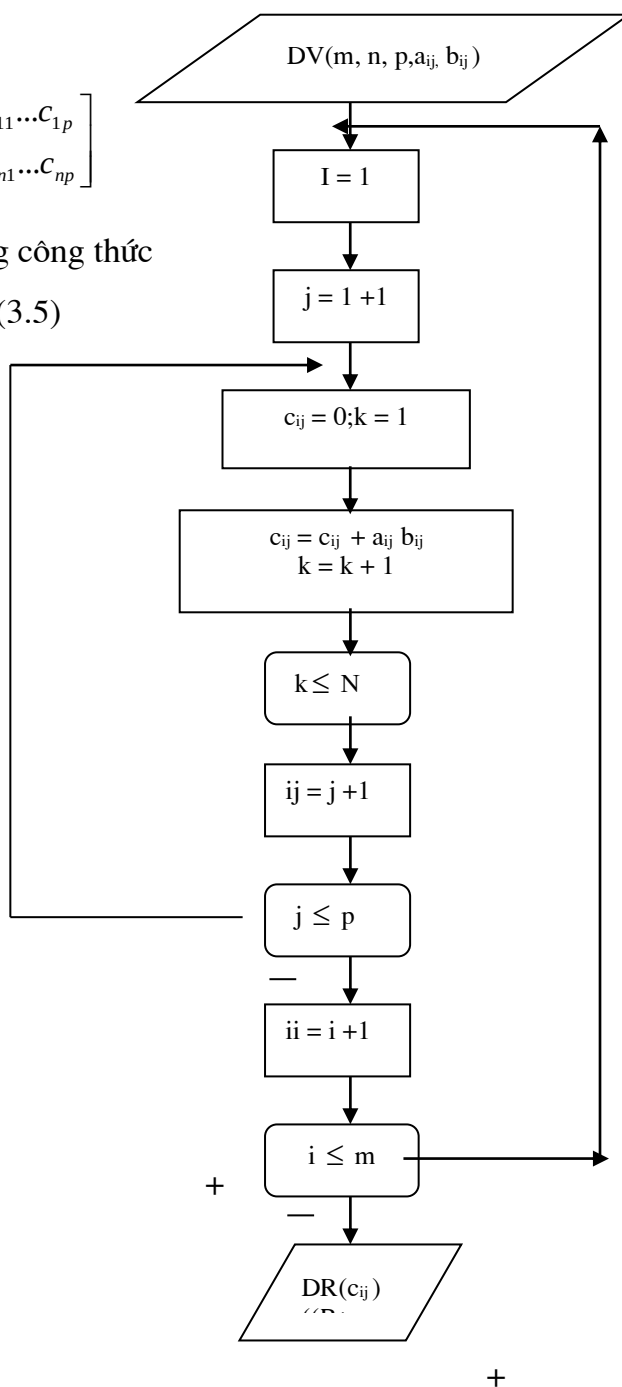
$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1p} \\ a_{n1} & \dots & a_{np} \end{bmatrix} \quad C = \begin{bmatrix} c_{11} & \dots & c_{1p} \\ c_{n1} & \dots & c_{np} \end{bmatrix}$$

Mỗi phần tử của ma trận C được xác định bằng công thức

$$C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}; i = 1 \div m; j = 1 \div p \quad (3.5)$$



Hình 3.15



Hình 3.16

Trong bài toán này, các phần tử của ma trận C chạy theo hai chỉ số i và j; bản thân mỗi phần tử  $c_{ij}$  lại là một bài toán tính tổng các tích  $a_{ik}b_{kj}$  với k chạy từ 1 đến n. Như vậy ta có ba chu trình lồng nhau. Chu trình trong cùng tính tổng  $c_{ij}$ . mỗi lần vào chu trình này phải gán lại giá trị ban đầu của nó bằng 0 ( $c_{ij} = 0$ ) và bộ đếm k bắt đầu lại từ  $k = 1$ . Ta có sơ đồ khối như hình 3.16.

### Thuật toán Xử lý bảng biểu, danh sách.

Cho danh sách sinh viên với kết quả học tập nh- sau ( ký hiệu ms là mã số , ht là họ tên, mon i là điểm môn thứ i , đtb là điểm trung bình, hk là mức hạnh kiểm , dhk là điểm hạnh kiểm, hb là tiền học bổng)

| msv | hoten | mon 1 |  | mon p | hk | dhk | đtb | hk |
|-----|-------|-------|--|-------|----|-----|-----|----|
|     |       |       |  |       |    |     |     |    |

trong đó :

- Cột msv là một mảng , mỗi phần tử là một xâu chín ký tự, gồm hai ký tự mã khoa, hai ký tự mã khoá học , dấu ngăn cách/, hai ký tự mã lớp, hai ký tự mã số thứ tự của sinh viên trong lớp;
- hoten là một mảng, mỗi phần tử là một xâu 30 ký tự chữ;
- mon là một mảng, mỗi phần tử là một số thực - điểm môn học , mỗi điểm bốn chữ số, mỗi phần tử là một ký tự chữ( có bốn mức hạnh kiểm A, B, C, D);
- đtb là một mảng, mỗi phần tử là một số thực - điểm trung bình chung mở rộng, sẽ đ- ọc tính trong quá trình thực hiện ch- ơng trình, theo công thức.

$$\text{đtb} = (\sum_{k=1}^p \text{mon}_k) / p + \text{dhk}_i ,$$

trong đó dhk đ- ọc tính nh- sau

$$\text{dhk} = \begin{cases} 0.6 \text{neula} \text{hk} \text{la} A \\ 0.3 \text{neula} \text{hk} \text{la} B \\ 0.6 \text{neula} \text{hk} \text{la} C \\ -0.3 \text{neula} \text{hk} \text{la} D \end{cases}$$

- hb - học bổng, đ- ọc tính nh- sau: tính suất học bổng trung bình.

$$\text{shb} = (\text{quỹ học bổng})/n,$$

n là số học sinh được cấp học bổng, bằng tổng số học sinh trừ đi những ng- ời bị kỷ luật không đ- ọc cấp học bổng;

- Tính  $y_1 = (\text{số sinh viên có đtb} < 5.0)/n$ ;

$$y_2 = (\text{số sinh viên có đtb từ } 5.0:7.0)/n; \quad (3.6)$$

$$y_3 = (\text{số sinh viên có đtb} > 7.0)/n;$$

Cho suất học bổng trung bình là 100%, ta phải xác định  $x_1, x_2, x_3$ , là phần trăm so với shb của từng loại  $y_1, y_2, y_3$ .

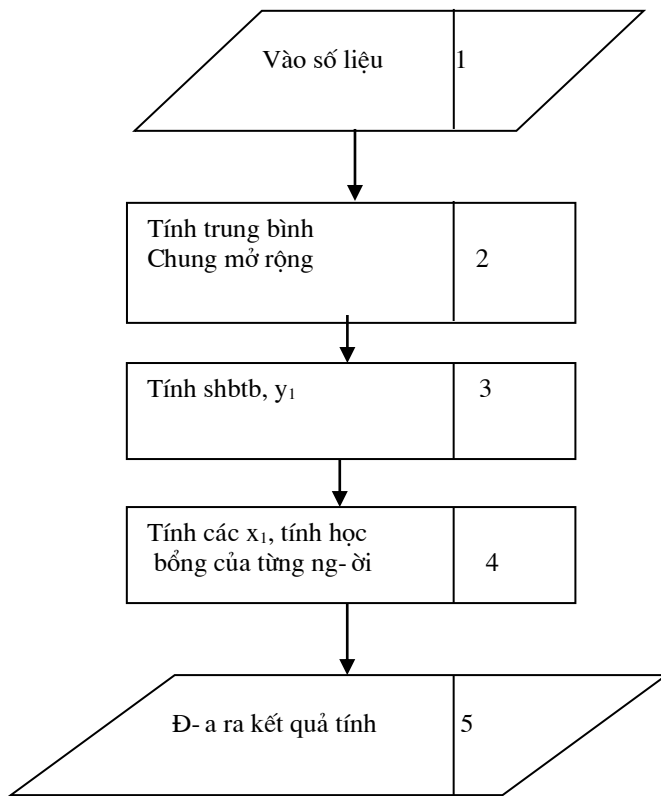
Để cân đối quỹ học bổng, phải xác định điều kiện thoả mãn ph- ơng trình

$$x_1 y_1 + x_2 y_2 + x_3 y_3 = 1 \quad (3.7)$$

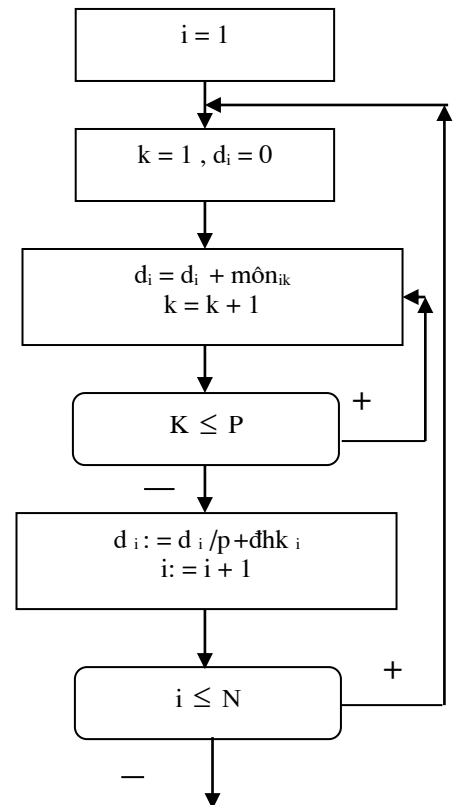
Ba hệ số  $y_1, y_2, y_3$  đ- ọc xác định từ (3.6), ta phải cho hai trong ba ẩn  $x_i$  giá trị thì mới xác định đ- ọc ẩn thứ ba, vì chỉ có một ph- ơng trình (3.7). Xuất phát từ nghiệp vụ quản lý sinh viên, ta cho  $x_1=80\%$ ;  $x_3=120\%$ . Thay các giá trị này vào (3.7) ta tìm đ- ọc  $x_2$ . nhận  $x_i$  với shb ta xác định đ- ọc tiền học bổng của từng loại sinh viên. Ta phân tích bài toán ra làm các khối lớn nh- trên hình 3.16.

Để làm ví dụ, ta triển khai một trong các khối trên, khối số 2.

Giả sử tr-ớc đó đã xác định điểm hạnh kiểm - đhk, điểm trung bình chung mở rộng ký hiệu là  $d_i$ . Hình 3.17 cho ta thuật toán



Hình 3.16



Hình 3.17

của khối 2. Trong thực tế, việc phân bổ học bổng cho toàn tr-ờng rất phức tạp, phải xét nhiều loại đối t-ợng, nhiều mức điểm khác nhau giữa các lớp, giữa các khoá, trong khuôn khổ chỉ tiêu học bổng hạn chế tùy theo từng năm; quỹ học bổng phải chia vừa hết, không đ-ợc thừa hoặc thiếu.

### Ví dụ 15. Hồi quy tuyến tính.

Phân tích hồi quy là ph-ơng pháp thống kê toán học nghiên cứu các mối liên hệ giữa các đại l-ợng ngẫu nhiên và không ngẫu nhiên, xác định hàm số biểu diễn các mối liên hệ đó.

Trong một quá trình thực nghiệm, ứng với các giá trị  $x_i$  ta thu đ-ợc  $y_i$ ,  $i=1÷n$ . Khi đ-ờng hồi quy thực nghiệm không quá gẫy khúc thì ta có thể phán đoán đ-ờng cong lý thuyết có dạng tuyến tính  $y=ax+b$ . Vấn đề đặt ra là phải xác định  $a$  và  $b$  sao cho.

$$\sum (\bar{y}_i - y_i)^2 \Rightarrow \min, \quad (3.8)$$

Trong đó  $\bar{y}_i$  là giá trị thực nghiệm;  $y_i$  là giá trị tính toán từ ph-ơng trình.

$$y_i = ax_i + b. \quad (3.9)$$

Theo lý thuyết, hàm  $\sum (\bar{y}_i - y_i)^2 = \sum (\bar{y}_i - ax_i - b)^2$ , đạt cực tiểu khi hàm bậc nhất theo  $a$  và  $b$  bằng 0

$$\sum (\bar{y}_i - ax_i - b) = 0; \quad (3.10)$$

$$\sum x_i (\bar{y}_i - ax_i - b) = 0$$

Giải hệ (4.12).

Từ phương trình thứ nhất ta rút a qua b

$$a = (\sum \bar{y}_i - nb) / (\sum x_i). \quad (3.11)$$

Thay a vào phương trình thứ hai, qua biến đổi, ta thu được

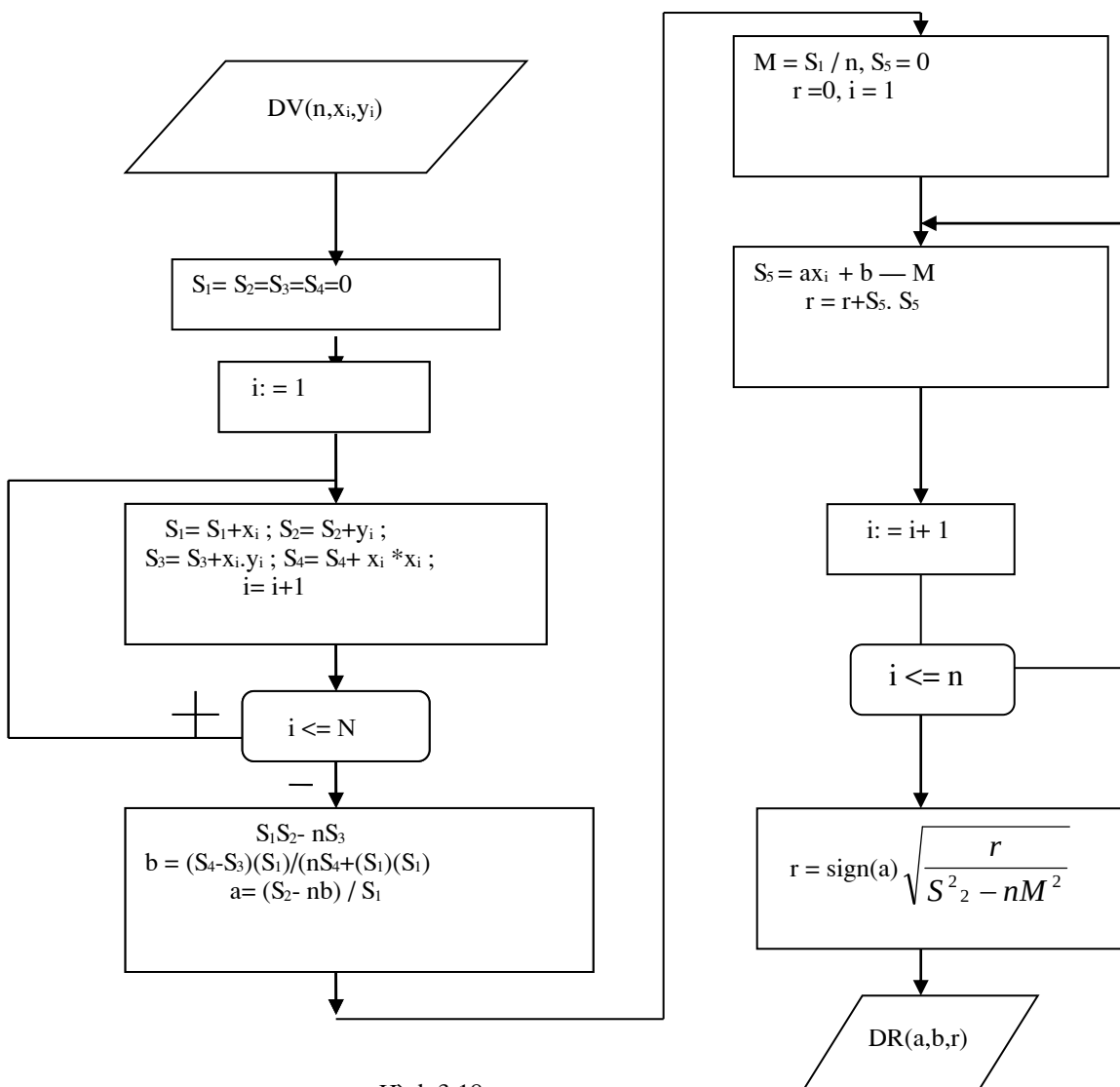
$$b = (\sum x_i^2 - \sum x_i \cdot \bar{y}_i) / [n \sum x_i^2 + (\sum x_i) (\sum x_i)].$$

Sau khi tìm được b, thay b vào (4.13) ta tìm được a

$$a = a = (\sum \bar{y}_i - nb) / (\sum x_i).$$

Sau khi tính được a, b ta cần xác định hệ số tương quan

$$r = \text{sign}(a) \sqrt{\left( \frac{\sum (y_i - M)}{\sum (y_i - M)^2} \right)^2} \quad (3.12)$$



Hình 3.18

Sơ đồ thuật toán nh- trên hình 3.18. (M là kỳ vọng toán học).

### 3.5.4. Thuật toán lặp có số vòng lặp ch- a biết tr- ớc

Trong thực tế có nhiều bài toán có chu trình không phải cho bằng số lần lặp mà là một biểu thức nào đó, chẳng hạn, sai số cho phép.

Ví dụ 2.16. Xét bài toán giải ph- ơng trình  $F(x) = 0$  bằng ph- ơng pháp lặp. Ta biến đổi ph- ơng trình  $F(x)=0$  về dạng  $f(x) = x$ . Với nghiệm ban đầu  $x_0$  ta thiết lập chuỗi.

$$x_1 = f(x_0),$$

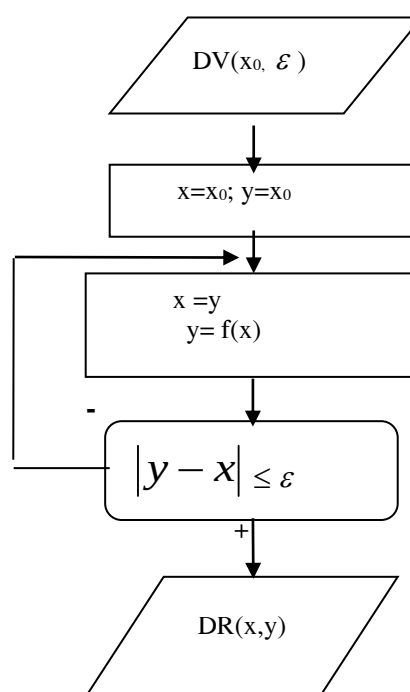
$$x_2 = f(x_1),$$

...

$x_n = f(x_{n-1})$ , quá trình dừng khi  $|x_n - x_{n-1}| \leq \varepsilon$ , với epsilon  $\varepsilon$  là giá trị cho tr- ớc.

số lần lặp phụ thuộc vào  $\varepsilon$

$\varepsilon$  càng bé thì số lần lặp càng nhiều



Hình 3.19

Hình 3.20

□ đây ta chỉ mới xét hàm  $f(x)$  ở dạng tổng quát.

Số lần tính lặp  $x_i = f(x_{i-1})$  phụ thuộc vào  $\varepsilon$ . Vì vậy ta không thể đặt n biến  $x_i$  (n ch- a xác định). Khi đó ta đặt một biến x nhận giá trị nghiệm ở mỗi thời điểm tính toán, một biến y nhận giá trị của hàm f; nếu trên mỗi b- ớc lặp ch- a thoả mãn biểu thức  $|x - y| \leq \varepsilon$  thì x sẽ nhận giá trị mới là  $(x:=y)$  cho tr- ớc lặp tiếp theo.

Hình 3.19 và 3.20 mô tả thuật toán bài toán trên theo hai cách kiểm tra điều kiện tr- ớc và sau. Khi tính cụ thể chỉ việc thay công thức của hàm số vào sơ đồ. Bạn đọc hãy làm bài tập với  $f(x) = x - \cos x + 0,7$ ;  $x_0=1$ ,  $\varepsilon=10^{-6}$ .

Ví dụ 17. Giải hệ ph- ơng trình đại số theo ph- ơng pháp Gauss.

Cho hệ phương trình đại số tuyến tính.

$$a_{1.1}X_1 + a_{1.2}X_2 + \dots + a_{1.n}X_n = a_{1.n+1}$$

$$a_{2.1}X_1 + a_{2.2}X_2 + \dots + a_{2.n}X_n = a_{2.n+1}$$

.....

$$a_{n.1}X_1 + a_{n.2}X_2 + \dots + a_{n.n}X_n = a_{n.n+1}$$

(1)

Theo phương pháp Gauss, từ hệ thức (1) ta đưa về dạng tam giác

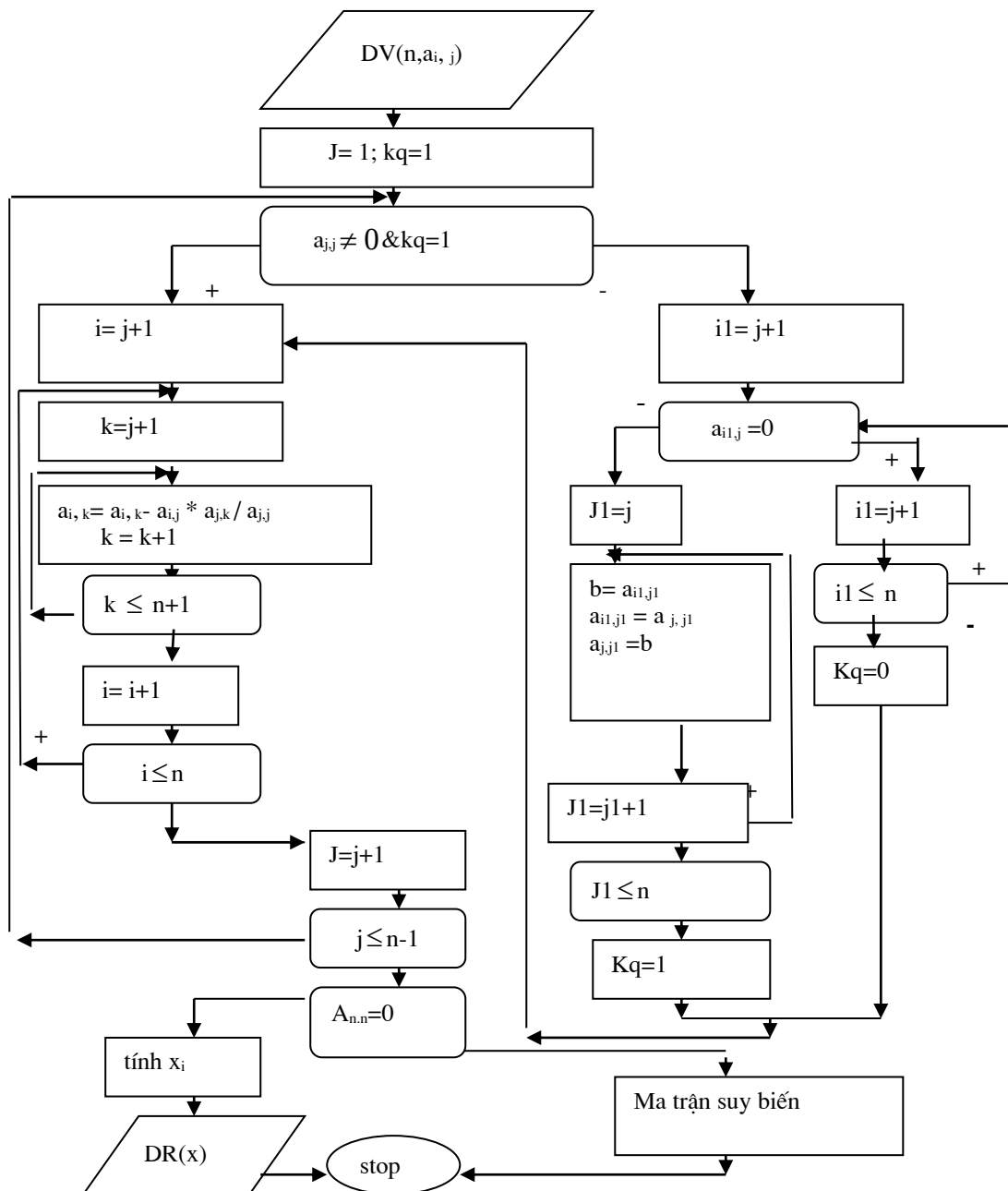
$$a_{1.1}X_1 + a_{1.2}X_2 + \dots + a_{1.n}X_n = a_{1.n+1}$$

$$a_{2.2}X_2 + \dots + a_{2.n}X_n = a_{2.n+1}$$

$$\dots = \dots$$

(2)

$$a_{n.n}X_n = a_{n.n+1}$$



Hình 3.21

Từ (2) ta sẽ tính ngược để tìm nghiệm:

$$x_n = a_{n,n+1} / a_{n,n}$$

$$x_{n-1} = (a_{n-1,n+1} - a_{n-1,n} * x_n) / a_{n-1,n-1} \dots$$

Như vậy thuật toán có hai giai đoạn: đưa ma trận ban đầu về dạng tam giác và giải ngược tìm nghiệm.

Giai đoạn thứ nhất

1) Bắt đầu từ  $j = 1$

2) Nếu  $a_{j,j} \neq 0$  thì dùng cách tổ hợp tuyến tính để khử các phần tử ở cột  $j$  kể từ hàng thứ  $j+1$  đến  $n$ , sau đó tăng  $j=j+1$ .

Nếu  $j \leq n-1$  thì trở lại mục 2), nếu  $j > n-1$  thì trở xuống mục 4).

3) Nếu  $a_{j,j} = 0$  phải tìm trong cột  $j$  một phần tử  $a_{k,j} \neq 0$  và đổi chỗ hai hàng  $k, j$  cho nhau;

4) Nếu  $a_{n,n} = 0$  thì suy ra ma trận  $A$  suy biến, hệ không có nghiệm duy nhất, dừng chương trình, nếu  $a_{n,n} \neq 0$ , xong giai đoạn thứ nhất.

Giai đoạn thứ hai

Tính các  $x_n, x_{n-1}, \dots$

Sơ đồ khối như trên hình 3.21.

## TỔNG KẾT CÁC DẠNG THUẬT TOÁN CƠ BẢN

| TT | Tên dạng            |                                      | Dùng câu lệnh                                                                                                            |
|----|---------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>Dạng tuần tự</b> |                                      | <b>Lệnh Gán =</b>                                                                                                        |
| 2  | <b>Rẽ nhánh</b>     | <b>2 nhánh</b>                       | <b>if, if ... else</b>                                                                                                   |
|    |                     | <b>Nhiều nhánh</b>                   | <b>switch... case</b>                                                                                                    |
| 3  | <b>Lặp</b>          | <b>Có số lần lặp biết trước</b>      | <b>for</b>                                                                                                               |
|    |                     | <b>Có số lần lặp chưa biết trước</b> | <b>do</b><br><b>{ các câu lệnh</b><br><b>} while(điều kiện)</b><br><br><b>while(điều kiện)</b><br><b>{ các câu lệnh}</b> |

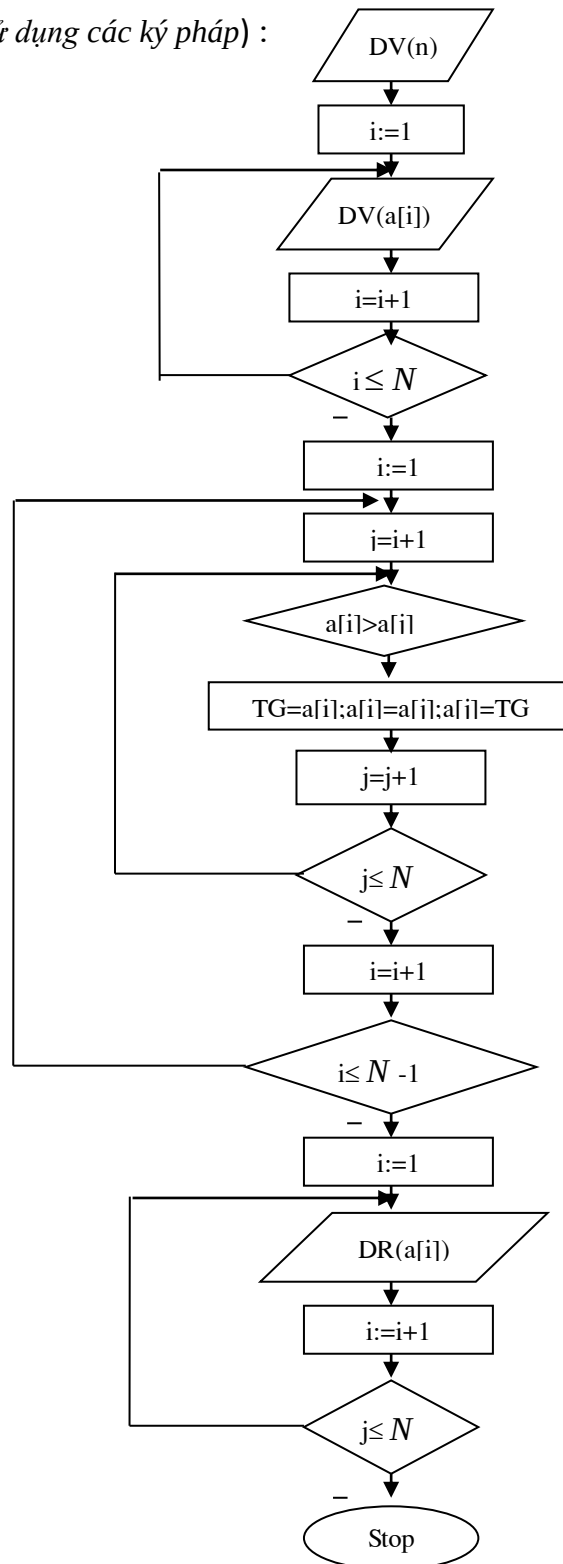


## CH- ƠNG 4. CÁC THUẬT TOÁN SẮP XẾP

### I. SẮP XẾP BẰNG CHỌN LỰA ĐƠN GIẢN CHO MẢNG

Bài toán : cho mảng  $a_1, \dots, a_n$  các số thực bất kỳ, lập thuật toán sắp xếp lại mảng theo thứ tự tăng dần.

Sơ đồ thuật toán (chú ý sử dụng các ký pháp) :



Bài tập : nếu không dùng biến  $tg$  thì viết lại các lệnh sau đây như thế nào:

$tg = a[i]; a[i] = a[j]; a[j] = tg;$  (không dùng hàm  $swap(a,b)$  có sẵn)

Ví dụ : nhập vào dãy số có số phần tử  $n=5$  rồi sắp xếp tăng dần.

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 9    | 2    | 7    | 4    | 5    |

$i=1$ , sẽ so sánh a[1] với a[2]...a[5], nếu có  $a[1]>a[j]$  thì hoán vị  $a[1] \longleftrightarrow a[j]$  ( $j=2 \div 5$ )

$J=i+1=2$  so sánh  $a[1]>a[j]$  thì hoán vị  $a[1] \longleftrightarrow a[j]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 9    | 7    | 4    | 5    |

Tiếp tục tăng  $j++$  và so sánh so sánh  $a[1]>a[3]$ , vì kết quả so sánh fail nên a[1] giữ nguyên.

Kết thúc vòng này a[1] là nhỏ nhất trong 5 số

Tiếp tục,  $i++$ ,

$I=2$ , sẽ so sánh a[2] với a[3]...a[5], nếu có  $a[2]>a[j]$  thì hoán vị  $a[2] \longleftrightarrow a[j]$  ( $j=3 \div 5$ )

$J=i+1=3$  so sánh  $a[2]>a[3]$  thì hoán vị  $a[2] \longleftrightarrow a[3]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 7    | 9    | 4    | 5    |

$J=i+1=4$  so sánh  $a[2]>a[4]$  thì hoán vị  $a[2] \longleftrightarrow a[4]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 9    | 7    | 5    |

$J=i+1=5$  so sánh  $a[2]>a[5]$  fail nên không hoán vị  $a[2] \longleftrightarrow a[5]$

Kết thúc vòng này a[2] là nhỏ nhất trong 4 số từ a[2] đến a[5]

$I=3$ , sẽ so sánh a[3] với a[4],a[5], nếu có  $a[3]>a[j]$  thì hoán vị  $a[3] \longleftrightarrow a[j]$  ( $j=4 \div 5$ )

$J=i+1=4$  so sánh  $a[3]>a[4]$  thì hoán vị  $a[3] \longleftrightarrow a[4]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 7    | 9    | 5    |

$J=i+1=5$  so sánh  $a[3]>a[5]$  thì hoán vị  $a[3] \longleftrightarrow a[5]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 5    | 9    | 7    |

Kết thúc vòng này a[3] là nhỏ nhất trong 3 số từ a[3] đến a[5]

$I=4$ , so sánh  $a[4]>a[5]$  thì hoán vị  $a[4] \longleftrightarrow a[5]$

$J=i+1=5$  so sánh  $a[4]>a[5]$  thì hoán vị  $a[4] \longleftrightarrow a[5]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 5    | 7    | 9    |

Kết thúc vòng này a[4] là nhỏ nhất trong 2 số từ a[4] đến a[5] và dãy số đã sắp xếp xong.

Đoạn code C++ của chương trình như sau :

```
for (i=1;i<n;i++)
{ for (j= i+1;j<= n;j++)
  { if (x[i] <x[j] ) { tg = x[i]; x[i] =x[j]; x[j]:= tg; } }
}
```

## II. SẮP XẾP CHO MẢNG LÀ MA TRẬN THỪA

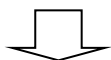
Ma trận thưa là dạng ma trận có rất ít phần tử có giá trị  $\neq 0$ . Khi nhập và xử lý chỉ xét các phần tử  $\neq 0$ .

Ví dụ : cho ma trận thưa A có m hàng, n cột, có q phần tử có giá trị  $\neq 0$ . Khi đó sẽ tổ chức lưu trữ dữ liệu trong một mảng có 3 cột và (q+1) hàng, ở đây q là số phần tử khác không. Hàng thứ nhất là m,n,q (số hàng, số cột, số phần tử khác 0 của ma trận ban đầu) :

| 0  | m   | n   | q                |
|----|-----|-----|------------------|
| 1  | i   | j   | $a[i][j] \neq 0$ |
| 2  |     |     |                  |
| .. | ... | ... | ...              |
| q  |     |     |                  |

Ví dụ . Cho ma trận thưa A có 6 hàng 6 cột, có 8 phần tử khác 0 :

|   | 1  | 2  | 3  | 4  | 5 | 6   |
|---|----|----|----|----|---|-----|
| 1 | 15 | 0  | 0  | 22 | 0 | -15 |
| 2 | 0  | 11 | 3  | 0  | 0 | 0   |
| 3 | 0  | 0  | 0  | -6 | 0 | 0   |
| 4 | 0  | 0  | 0  | 0  | 0 | 0   |
| 5 | 91 | 0  | 0  | 0  | 0 | 0   |
| 6 | 0  | 0  | 28 | 0  | 0 | 0   |



Ta nhập theo mảng 9 hàng 3 cột :

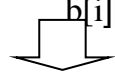
| A | [0 | 6i | 6j | 8a[i][j]#0 |
|---|----|----|----|------------|
|   | [1 | 1  | 1  | 15         |
|   | [2 | 1  | 4  | 22         |
|   | [3 | 1  | 6  | -15        |
|   | [4 | 2  | 2  | 11         |
|   | [5 | 2  | 3  | 3          |
|   | [6 | 3  | 4  | -6         |
|   | [7 | 5  | 1  | 91         |

[8                  6                  3                  28

**Hoán vị ma trận :** khi hoán vị, ta chuyển hàng thành cột và ng- ọc lại .

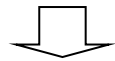
$$b[i][j] = a[j][i] \text{ khi } i \neq j$$

$$b[i][j] = a[j][i] \text{ khi } i=j$$



|           |    |   |   |     |
|-----------|----|---|---|-----|
| $B = A^T$ | [0 | 6 | 6 | 8   |
|           | [1 | 1 | 1 | 15  |
|           | [2 | 4 | 1 | 22  |
|           | [3 | 6 | 1 | -15 |
|           | [4 | 2 | 2 | 11  |
|           | [5 | 3 | 2 | 3   |
|           | [6 | 4 | 3 | -6  |
|           | [7 | 1 | 5 | 91  |
|           | [8 | 3 | 6 | 28  |

Sau đó phải sắp xếp lại mảng 3 cột, (q+1) hàng theo trật tự tăng dần cột chứa hàng i của B :



|                     |     |       |       |             |
|---------------------|-----|-------|-------|-------------|
|                     |     | $1-i$ | $2-j$ | $3-a[i][j]$ |
|                     | B[0 | 6     | 6     | 8           |
| $B=A^T \Rightarrow$ | B[1 | 1     | 1     | 15          |
|                     | B[2 | 1     | 5     | 91          |
|                     | B[3 | 2     | 2     | 11          |
|                     | B[4 | 3     | 2     | 3           |
|                     | B[5 | 3     | 6     | 28          |
|                     | B[6 | 4     | 1     | 22          |
|                     | B[7 | 4     | 3     | -6          |
|                     | B[8 | 6     | 1     | -15         |

**Bài tập :** Dựa theo ý tưởng xử lý ma trận thừa như trên hãy vẽ sơ đồ thuật toán và viết chương trình C++ nhập, hoán vị ma trận thừa khi ma trận được lưu và xử lý ở dạng mảng có 3 cột, q+1 hàng (q là số phần tử  $\neq 0$ ).

### III. SẮP XẾP KIỂU NỔI BỌT - SẮP XẾP DẦN – BUBBLE SORT

Sắp xếp nổi bọt theo trật tự tăng dần đ-ợc thực hiện theo thuật toán : trong mỗi lần đi qua các dữ liệu nếu không có thứ tự đúng thì sắp xếp ngay hai phần tử liền kề . Theo ph-ơng pháp này, mỗi lần duyệt qua một l-ợt cả danh sách, một số phần tử lớn đ-ợc nổi lên trên, một số phần tử bé bị chìm xuống d-ới. Nếu sắp xếp theo thứ tự giảm dần thì một số phần tử lớn bị chìm xuống d-ới, một số phần tử bé đ-ợc nổi lên trên.

Xét bài toán : cho dãy các số thực  $a_1, a_2, a_3, \dots, a_n$  , yêu cầu sắp xếp lại theo trật tự tăng dần theo kiểu nổi bọt.

Bắt đầu từ so sánh  $a_1$  và  $a_2$ . Nếu  $a_1 > a_2$  thì hoán đổi giá trị giữa  $a_1$  và  $a_2$

Tiếp tục so sánh  $a_2$  với  $a_3$  Nếu  $a_2 > a_3$  thì hoán đổi giá trị giữa  $a_2$  và  $a_3$  . .

Tiếp tục so sánh  $a_i$  với  $a_{i+1}$  cho đến  $a_{n-1}$  và  $a_n$

Nh- vậy kết thúc lần đi qua thứ nhất, phần tử lớn nhất đã “nổi “ lên cuối danh sách,  $a[n]$  là lớn nhất, và một vài phần tử đã “chìm ” về phía gần đầu danh sách.

Ví dụ ,cho dãy ban đầu :

|        |        |     |          |        |
|--------|--------|-----|----------|--------|
| $a[1]$ | $a[2]$ | ... | $a[n-1]$ | $a[n]$ |
|--------|--------|-----|----------|--------|

Sau vòng lặp thứ nhất,  $a[n]$  là lớn nhất, nên không xét ở vòng lặp tiếp theo.

|        |        |     |          |
|--------|--------|-----|----------|
| $a[1]$ | $a[2]$ | ... | $a[n-1]$ |
|--------|--------|-----|----------|

Sau vòng lặp thứ hai,  $a[n-1]$  là lớn nhất, nên không xét ở vòng lặp tiếp theo.

|        |        |     |          |
|--------|--------|-----|----------|
| $a[1]$ | $a[2]$ | ... | $a[n-2]$ |
|--------|--------|-----|----------|

Tiếp tục cho đến vòng lặp thứ  $n-1$ , chỉ còn lại hai số, so sánh và hoán vị nếu cần.

|        |        |
|--------|--------|
| $a[1]$ | $a[2]$ |
|--------|--------|

Như vậy, có 2 vòng lặp lồng nhau :

Vòng ngoài chạy theo  $i$  từ 1 đến  $n-1$

Vòng trong chạy theo  $j$  từ 1 đến  $n-i$  , thực hiện phép so sánh nếu  $a[j] > a[j+1]$  thì hoán vị  $a[j] \longleftrightarrow a[j+1]$ , khi  $i=n-1; j=n-i=n-(n-1)=n-n+1=1$

Tiếp tục quá trình này, các phần tử lớn nhất trong danh sách con còn lại sẽ ở vào cuối cùng của danh sách con đó.

Ví dụ : nhập vào dãy số rồi sắp xếp tăng dần.

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| $a[1]$ | $a[2]$ | $a[3]$ | $a[4]$ | $a[5]$ |
| 9      | 2      | 7      | 4      | 5      |

$i=1$ ; lần lượt so sánh 2 phần tử kế tiếp, phần tử lớn hơn được hoán vị ra sau .

$J=1$  so sánh  $a[j] > a[j+1]$  thì hoán vị  $a[1] \longleftrightarrow a[2]$

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| $a[1]$ | $a[2]$ | $a[3]$ | $a[4]$ | $a[5]$ |
| 2      | 9      | 7      | 4      | 5      |

Tiếp tục tăng  $j++$  và so sánh  $a[j] > a[j+1]$ , hoán vị  $a[2] \leftrightarrow a[3]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 7    | 9    | 4    | 5    |

Tiếp tục tăng  $j++=3$  và so sánh  $a[j] > a[j+1]$ , hoán vị  $a[3] \leftrightarrow a[4]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 7    | 4    | 9    | 5    |

Tiếp tục tăng  $j++=4$  và so sánh  $a[j] > a[j+1]$ , hoán vị  $a[4] \leftrightarrow a[5]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 7    | 4    | 5    | 9    |

Kết thúc vòng này  $a[5]$  là lớn nhất trong 5 số

Tiếp tục  $i=2$

$j=1$  so sánh  $a[j] > a[j+1]$  fail nên không hoán vị

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 7    | 4    | 5    | 9    |

Tiếp tục tăng  $j++$  và so sánh  $a[j] > a[j+1]$ , hoán vị  $a[2] \leftrightarrow a[3]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 7    | 5    | 9    |

Tiếp tục tăng  $j++$  và so sánh  $a[j] > a[j+1]$ , hoán vị  $a[3] \leftrightarrow a[4]$

| a[1] | a[2] | a[3] | a[4] | a[5] |
|------|------|------|------|------|
| 2    | 4    | 5    | 7    | 9    |

Kết thúc vòng này  $a[4]$  là lớn nhất trong 4 số

Tiếp tục  $i=3$ , lặp lại  $j=1 \rightarrow 3$  vì các so sánh đều SAI nên không hoán vị,  $a[3]=5$  lớn nhất

Tiếp tục  $i=4$ , lặp lại  $j=1 \rightarrow 2$  vì các so sánh đều SAI nên không hoán vị,  $a[2]=4$  lớn nhất

Kết thúc.

Tổng quát : khi  $i=1$ ;  $j$  chạy từ  $1 \rightarrow n-1$ , so sánh từng cặp  $a[j] > a[j+1]$  nếu đúng (true) thì hoán vị  $a[j]$  với  $a[j+1]$ , kết thúc, ta khẳng định  $a[n]$  là lớn nhất.

Lặp lại quá trình trên, mỗi lần tăng  $i=i+1$ ,  $j$  từ 1 đến  $n-i$ , so sánh  $a[j] > a[j+1]$  nếu đúng (true) thì hoán vị  $a[j]$  với  $a[j+1]$ , kết thúc, ta khẳng định  $a[n-i]$  là lớn nhất.

Như vậy thuật toán có 2 vòng lặp lồng nhau, vòng ngoài theo  $i$  chạy từ 1 đến  $n-1$ , vòng trong  $j$  chạy từ 1 đến  $n-i$ .

**Bài tập về nhà :** vẽ sơ đồ thuật toán và viết chương trình C++ sắp xếp lại dãy số nhập từ bàn phím theo chiều giá trị tăng dần bằng cách sắp xếp nổi bọt.